SIMATS ENGINEERING

ASSESSMENT TOOL 2

COURSE NAME: Database Management Systems

COURSE CODE: CSA0504

COURSE OUTCOME COVERED

CO3: ***Analyze relational databases using functional dependencies, normalization (1NF to 5NF), and key constraints to identify redundancy and ensure data integrity.*** **(BL2, BL3)**

Activity Tool : Case Study (Medium)

Assessment Tool Weightage: 30%

QUESTIONS			**Total Marks: 50**
Q.No	**Question**	**Bloom's Level**	**Marks**
1	**Analyze the case: An online shopping platform stores Order(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price). Identify FDs and normalize to 3NF.**	**BL2 – Understand**	**5**
2	**A hospital maintains Patient(PID, PName, DocID, DocName, Specialization, WardNo, WardName). Identify anomalies and normalize to BCNF.**	**BL3 – Apply**	**5**
3	**Case study: A university stores Enrollment(SID, SName, CourseID, CourseName, Faculty, Grade). Analyze FDs and apply 2NF decomposition.**	**BL3 – Apply**	**5**
4	**Given the airline booking case: Booking(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date). Normalize up to BCNF.**	**BL3 – Apply**	**5**

5	**Analyze an HR system schema for a multinational company. Identify all functional and transitive dependencies and normalize to 3NF.**	**BL2 – Understand**	**5**
6	**Case: Library(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate). Identify MVDs and decompose to 4NF.**	**BL3 – Apply**	**5**
7	**Analyze a telecom billing schema and identify all candidate keys, primary keys, and functional dependencies before normalization.**	**BL2 – Understand**	**5**
8	**A retail inventory system has Product(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse). Apply complete normalization.**	**BL3 – Apply**	**5**
9	**Study the given poorly normalized schema for a School ERP and propose a fully normalized design up to BCNF with justification.**	**BL3 – Apply**	**5**
10	**Given a movie streaming platform schema, identify join dependencies and decompose to 5NF with lossless-join verification.**	**BL3 – Apply**	**5**

Code of Conduct:

I B Aishwarya/192421432(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.

I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student: B Aishwarya

RUBRICS FOR CALCULATION:

Criteria	Level 4 – Exemplary	Level 3 – Proficient	Level 2 – Developing	Level 1 – Beginning
Understanding of Case	Thorough understanding; clearly identifies all key issues	Good understanding with most issues identified	Basic understanding; some issues missed	Poor understanding; problem not identified correctly
Application of Concepts	Effectively applies relevant concepts to analyze the case deeply	Applies appropriate concepts with minor gaps	Limited application of concepts	Fails to apply relevant concepts
Analysis	In-depth analysis with strong reasoning and insights	Good analysis with logical reasoning	Basic analysis with limited depth	Weak or no analysis; lacks reasoning
Solution Design	Innovative, feasible solutions with strong justification	Logical solutions with adequate justification	Basic solutions with weak justification	Incorrect or no solution provided
Clarity	Clear, well-structured, and easy to understand	Organized and understandable presentation	Limited clarity and structure	Poor clarity; difficult to understand

Q1. Online Shopping Platform – Normalize to 3NF

Given Relation

ORDER(OrderID, CustID, CustName, ProdID, ProdName, Qty, Price)

Functional Dependencies

1. CustID → CustName
2. ProdID → ProdName, Price
3. (OrderID, ProdID) → CustID, Qty
4. Therefore, (OrderID, ProdID) is the candidate key.

Problems

There are partial dependencies:

- CustID → CustName
- ProdID → ProdName, Price

These cause redundancy.

3NF Decomposition

CUSTOMER

CUSTOMER(CustID, CustName)

- PK: CustID

PRODUCT

PRODUCT(ProdID, ProdName, Price)

- PK: ProdID

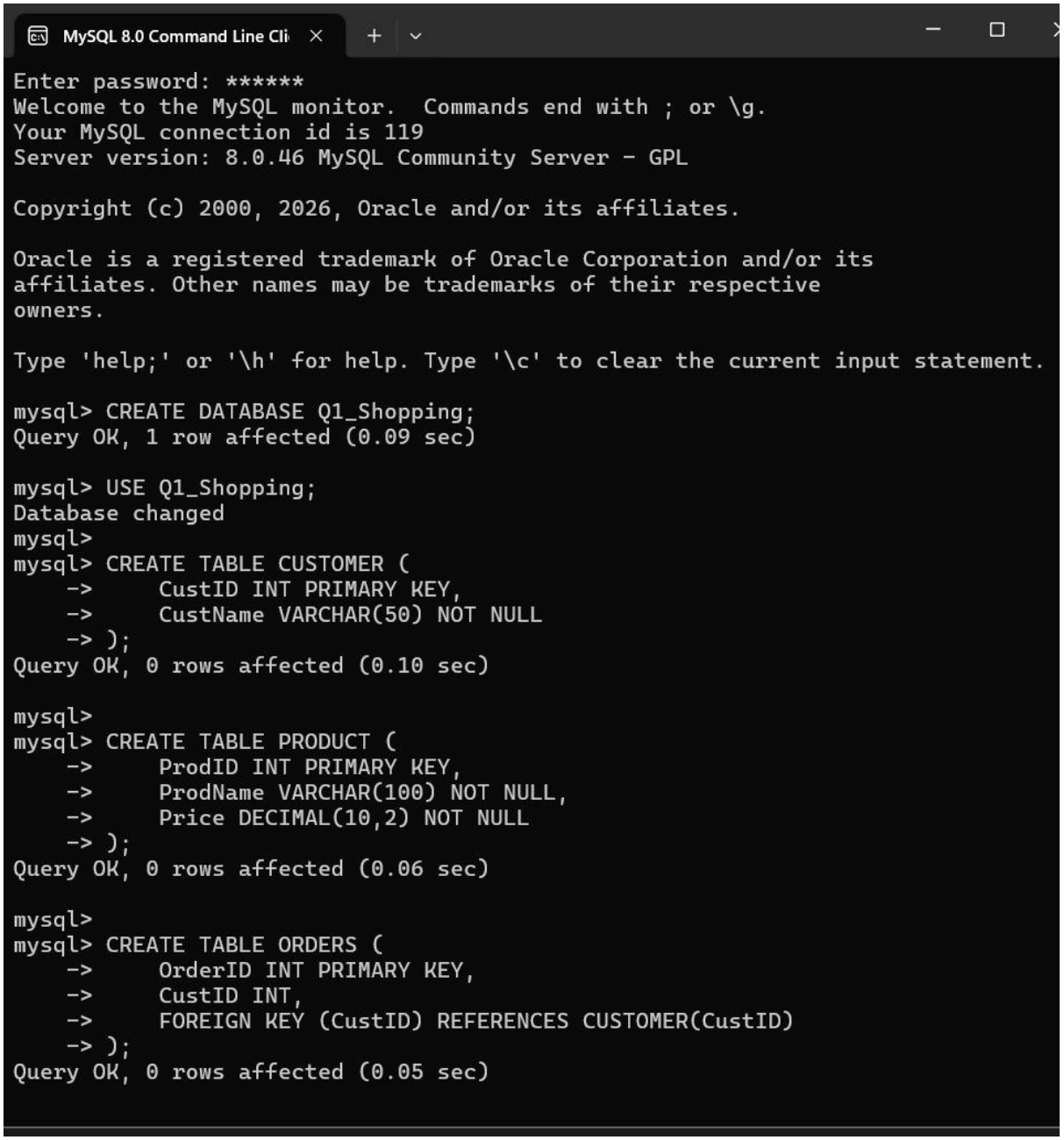
ORDER_ITEM

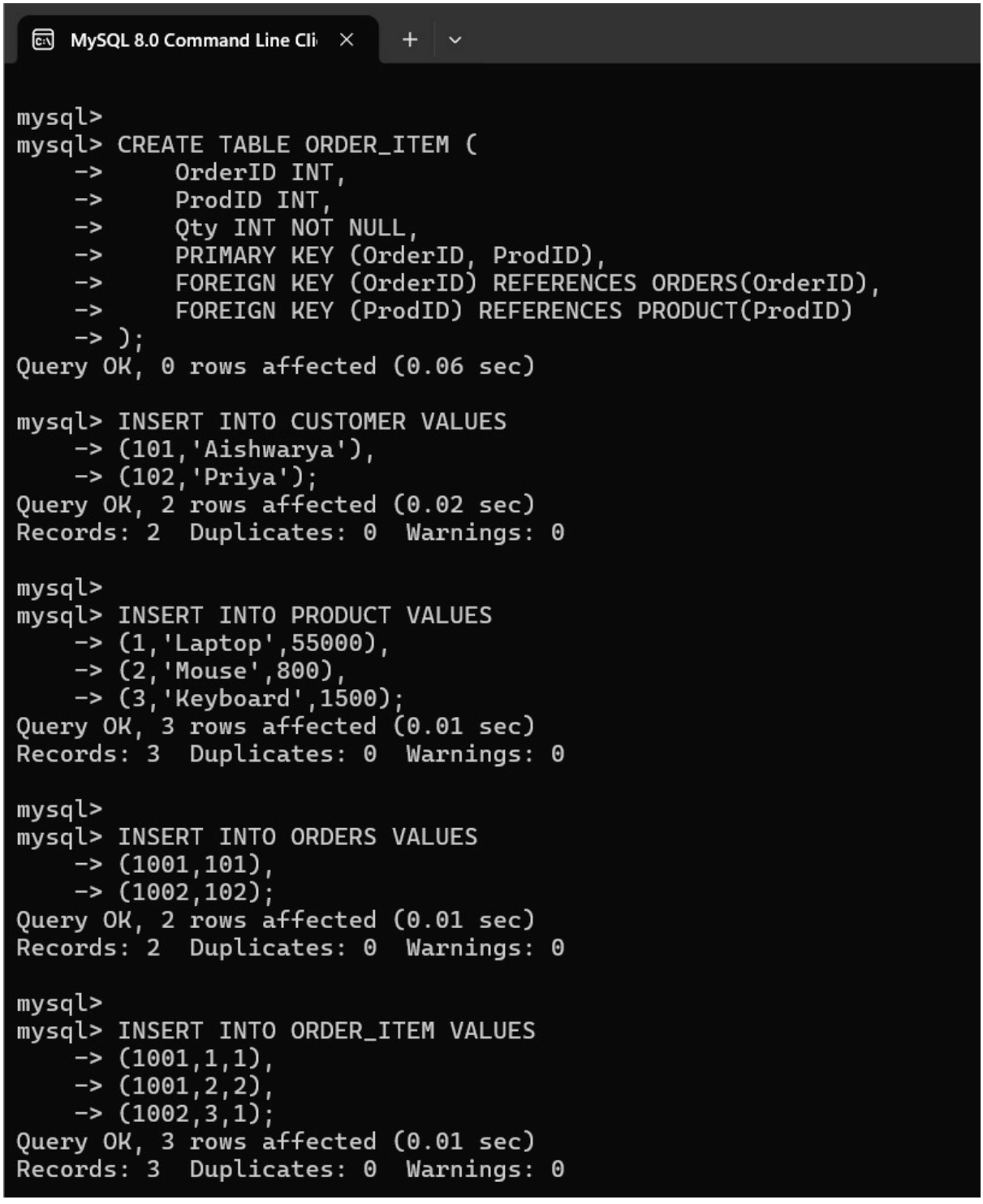
ORDER_ITEM(OrderID, ProdID, CustID, Qty)

- PK: (OrderID, ProdID)
- FKs: CustID, ProdID

Result

The decomposition removes update, insertion and deletion anomalies and produces a **3NF design**.


MySQL 8.0 Command Line Cli
Enter password: ******
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 119
Server version: 8.0.46 MySQL Community Server - GPL
Copyright (c) 2000, 2026, Oracle and/or its affiliates.
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
mysql> CREATE DATABASE Q1_Shopping;
Query OK, 1 row affected (0.09 sec)
mysql> USE Q1_Shopping;
Database changed
mysql>
mysql> CREATE TABLE CUSTOMER (
    ->   CustID INT PRIMARY KEY,
    ->   CustName VARCHAR(50) NOT NULL
    ->);
Query OK, 0 rows affected (0.10 sec)
mysql>
mysql> CREATE TABLE PRODUCT (
    ->   ProdID INT PRIMARY KEY,
    ->   ProdName VARCHAR(100) NOT NULL,
    ->   Price DECIMAL(10,2) NOT NULL
    ->);
Query OK, 0 rows affected (0.06 sec)
mysql>
mysql> CREATE TABLE ORDERS (
    ->   OrderID INT PRIMARY KEY,
    ->   CustID INT,
    ->   FOREIGN KEY (CustID) REFERENCES CUSTOMER(CustID)
    ->);
Query OK, 0 rows affected (0.05 sec)


MySQL 8.0 Command Line Cli
mysql>
mysql> CREATE TABLE ORDER_ITEM (
    ->   OrderID INT,
    ->   ProdID INT,
    ->   Qty INT NOT NULL,
    ->   PRIMARY KEY (OrderID, ProdID),
    ->   FOREIGN KEY (OrderID) REFERENCES ORDERS(OrderID),
    ->   FOREIGN KEY (ProdID) REFERENCES PRODUCT(ProdID)
    ->);
Query OK, 0 rows affected (0.06 sec)
mysql> INSERT INTO CUSTOMER VALUES
    -> (101,'Aishwarya'),
    -> (102,'Priya');
Query OK, 2 rows affected (0.02 sec)
Records: 2 Duplicates: 0 Warnings: 0
mysql>
mysql> INSERT INTO PRODUCT VALUES
    -> (1,'Laptop',55000),
    -> (2,'Mouse',800),
    -> (3,'Keyboard',1500);
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0
mysql>
mysql> INSERT INTO ORDERS VALUES
    -> (1001,101),
    -> (1002,102);
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0
mysql>
mysql> INSERT INTO ORDER_ITEM VALUES
    -> (1001,1,1),
    -> (1001,2,2),
    -> (1002,3,1);
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0

MySQL 8.0 Command Line Cli

```
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM CUSTOMER;
+--------+-----------+
| CustID | CustName  |
+--------+-----------+
|    101 | Aishwarya |
|    102 | Priya     |
+--------+-----------+
2 rows in set (0.01 sec)

mysql> SELECT * FROM PRODUCT;
+--------+----------+----------+
| ProdID | ProdName | Price    |
+--------+----------+----------+
|      1 | Laptop   | 55000.00 |
|      2 | Mouse    |   800.00 |
|      3 | Keyboard |  1500.00 |
+--------+----------+----------+
3 rows in set (0.00 sec)

mysql> SELECT * FROM ORDERS;
+---------+--------+
| OrderID | CustID |
+---------+--------+
|    1001 |    101 |
|    1002 |    102 |
+---------+--------+
2 rows in set (0.00 sec)

mysql> SELECT * FROM ORDER_ITEM;
+---------+--------+-----+
| OrderID | ProdID | Qty |
+---------+--------+-----+
|    1001 |      1 |   1 |
|    1001 |      2 |   2 |
|    1002 |      3 |   1 |
+---------+--------+-----+
3 rows in set (0.00 sec)
```

Q2. Hospital Database – Normalize to BCNF

Given Relation

PATIENT(PID, PName, DocID, DocName, Specialization, WardNo, WardName)

Functional Dependencies

1. PID → PName, DocID, WardNo
2. DocID → DocName, Specialization
3. WardNo → WardName

Candidate key:

PID

because:

PID+ = {PID, PName, DocID, DocName, Specialization, WardNo, WardName}

Anomalies

- **Update anomaly:** Doctor information is repeated for many patients.
- **Insertion anomaly:** A doctor cannot easily be stored without a patient.
- **Deletion anomaly:** Deleting the last patient may remove doctor/ward information.

BCNF Decomposition

PATIENT

PATIENT(PID, PName, DocID, WardNo)

DOCTOR

DOCTOR(DocID, DocName, Specialization)

WARD

WARD(WardNo, WardName)

BCNF Check

- In PATIENT, PID is the key.
- In DOCTOR, DocID is the key.
- In WARD, WardNo is the key.

Every determinant is a **superkey**.

Result

The relations are in **BCNF**, with reduced redundancy and improved integrity.

MySQL 8.0 Command Line Cli

```
    -> );
Query OK, 0 rows affected (0.06 sec)

mysql> INSERT INTO DOCTOR VALUES
    -> (1,'Dr. Kumar','Cardiology'),
    -> (2,'Dr. Ravi','Neurology');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO WARD VALUES
    -> (101,'General Ward'),
    -> (102,'ICU');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO PATIENT VALUES
    -> (1001,'Aishwarya',1,101),
    -> (1002,'Priya',2,102);
Query OK, 2 rows affected (0.02 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM DOCTOR;
+-------+-----------+----------------+
| DocID | DocName   | Specialization |
+-------+-----------+----------------+
|     1 | Dr. Kumar | Cardiology     |
|     2 | Dr. Ravi  | Neurology      |
+-------+-----------+----------------+
2 rows in set (0.00 sec)

mysql> SELECT * FROM WARD;
+--------+--------------+
| WardNo | WardName     |
+--------+--------------+
|    101 | General Ward |
|    102 | ICU          |
+--------+--------------+
2 rows in set (0.03 sec)
```

```
mysql> SELECT * FROM PATIENT;
+------+-----------+-------+--------+
| PID  | PName     | DocID | WardNo |
+------+-----------+-------+--------+
| 1001 | Aishwarya |     1 |    101 |
| 1002 | Priya     |     2 |    102 |
+------+-----------+-------+--------+
2 rows in set (0.00 sec)
```

Q3. University Enrollment – Apply 2NF

Given Relation

ENROLLMENT(SID, SName, CourseID, CourseName, Faculty, Grade)

Functional Dependencies

1. SID → SName
2. CourseID → CourseName, Faculty
3. (SID, CourseID) → Grade

Candidate Key

(SID, CourseID)

because a student's grade is determined by the combination of student and course.

Partial Dependencies

Since the key is composite:

- SID → SName
- CourseID → CourseName, Faculty

are partial dependencies.

Therefore, the relation is not in 2NF.

2NF Decomposition

STUDENT

STUDENT(SID, SName)

COURSE

COURSE(CourseID, CourseName, Faculty)

ENROLLMENT

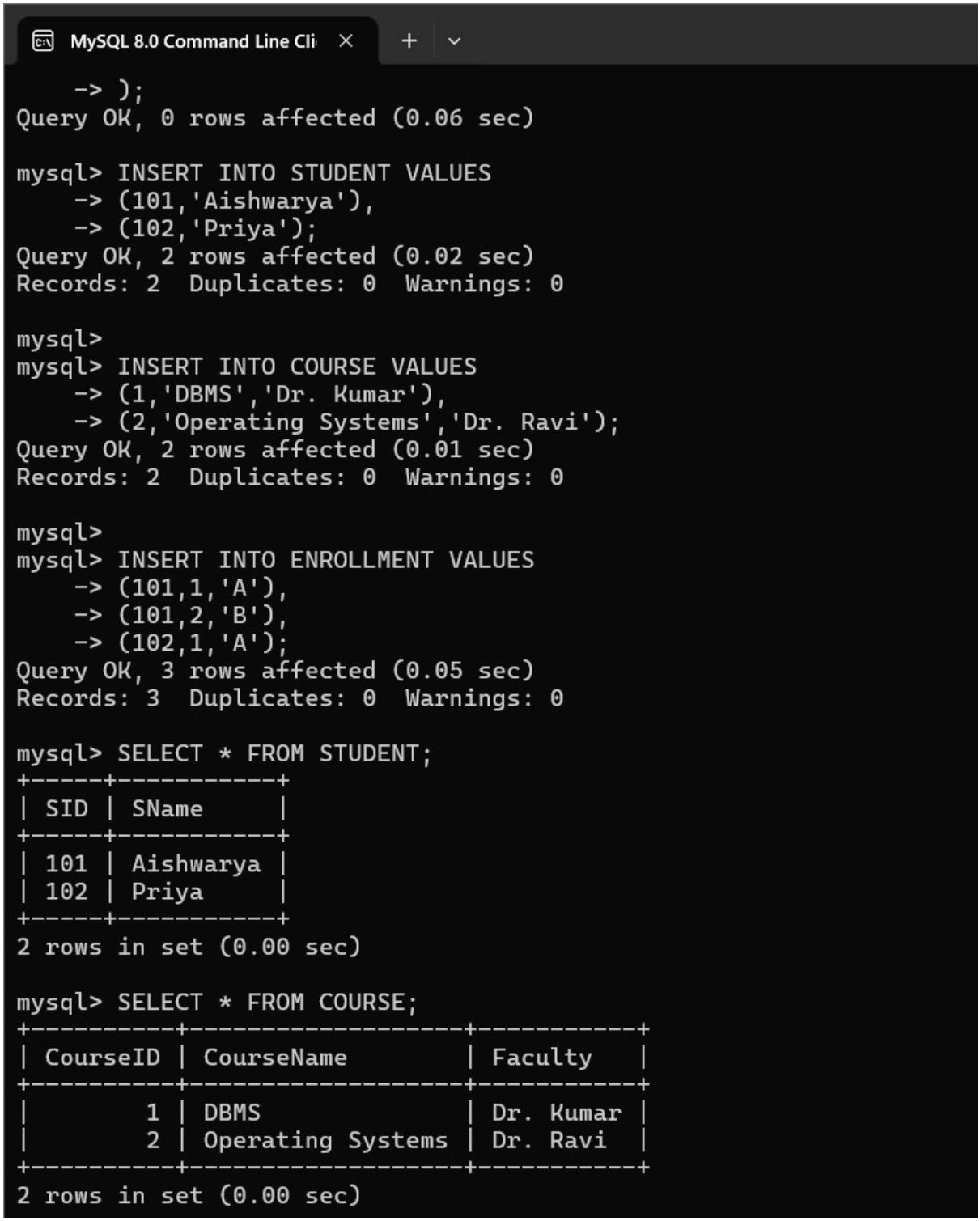
ENROLLMENT(SID, CourseID, Grade)

Keys

- STUDENT: SID
- COURSE: CourseID
- ENROLLMENT: (SID, CourseID)

Result

All partial dependencies are removed, so the design is in **2NF**.


MySQL 8.0 Command Line Cli
->);
Query OK, 0 rows affected (0.06 sec)
mysql> INSERT INTO STUDENT VALUES
-> (101,'Aishwarya'),
-> (102,'Priya');
Query OK, 2 rows affected (0.02 sec)
Records: 2 Duplicates: 0 Warnings: 0
mysql>
mysql> INSERT INTO COURSE VALUES
-> (1,'DBMS','Dr. Kumar'),
-> (2,'Operating Systems','Dr. Ravi');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0
mysql>
mysql> INSERT INTO ENROLLMENT VALUES
-> (101,1,'A'),
-> (101,2,'B'),
-> (102,1,'A');
Query OK, 3 rows affected (0.05 sec)
Records: 3 Duplicates: 0 Warnings: 0
mysql> SELECT * FROM STUDENT;
+-----+-----------+
| SID | SName |
+-----+-----------+
| 101 | Aishwarya |
| 102 | Priya |
+-----+-----------+
2 rows in set (0.00 sec)
mysql> SELECT * FROM COURSE;
+----------+------------------+-----------+
| CourseID | CourseName | Faculty |
+----------+------------------+-----------+
| 1 | DBMS | Dr. Kumar |
| 2 | Operating Systems | Dr. Ravi |
+----------+------------------+-----------+
2 rows in set (0.00 sec)

```
mysql> SELECT * FROM ENROLLMENT;
+-----+----------+-------+
| SID | CourseID | Grade |
+-----+----------+-------+
| 101 |        1 | A     |
| 101 |        2 | B     |
| 102 |        1 | A     |
+-----+----------+-------+
3 rows in set (0.00 sec)

mysql> SELECT S.SID, S.SName, C.CourseID, C.CourseName,
    ->        C.Faculty, E.Grade
    -> FROM STUDENT S
    -> JOIN ENROLLMENT E ON S.SID = E.SID
    -> JOIN COURSE C ON E.CourseID = C.CourseID;
+-----+-----------+----------+------------------+-----------+-------+
| SID | SName     | CourseID | CourseName       | Faculty   | Grade |
+-----+-----------+----------+------------------+-----------+-------+
| 102 | Priya     |        1 | DBMS             | Dr. Kumar | A     |
| 101 | Aishwarya |        1 | DBMS             | Dr. Kumar | A     |
| 101 | Aishwarya |        2 | Operating Systems | Dr. Ravi  | B     |
+-----+-----------+----------+------------------+-----------+-------+
3 rows in set (0.05 sec)
```

Q4. Airline Booking – Normalize up to BCNF

Given Relation

BOOKING(FlightNo, PassID, PassName, Seat, DeptCity, ArrCity, Date)

Assume a flight on a particular date identifies its route and a passenger's booking identifies the assigned seat.

Functional Dependencies

1. PassID → PassName
2. FlightNo → DeptCity, ArrCity
3. (FlightNo, Date, PassID) → Seat

Candidate key:

(FlightNo, Date, PassID)

1NF

All attributes contain atomic values, so the relation is in 1NF.

2NF

There are partial dependencies:

- PassID → PassName
- FlightNo → DeptCity, ArrCity

Decompose into:

PASSENGER

PASSENGER(PassID, PassName)

FLIGHT

FLIGHT(FlightNo, DeptCity, ArrCity)

BOOKING

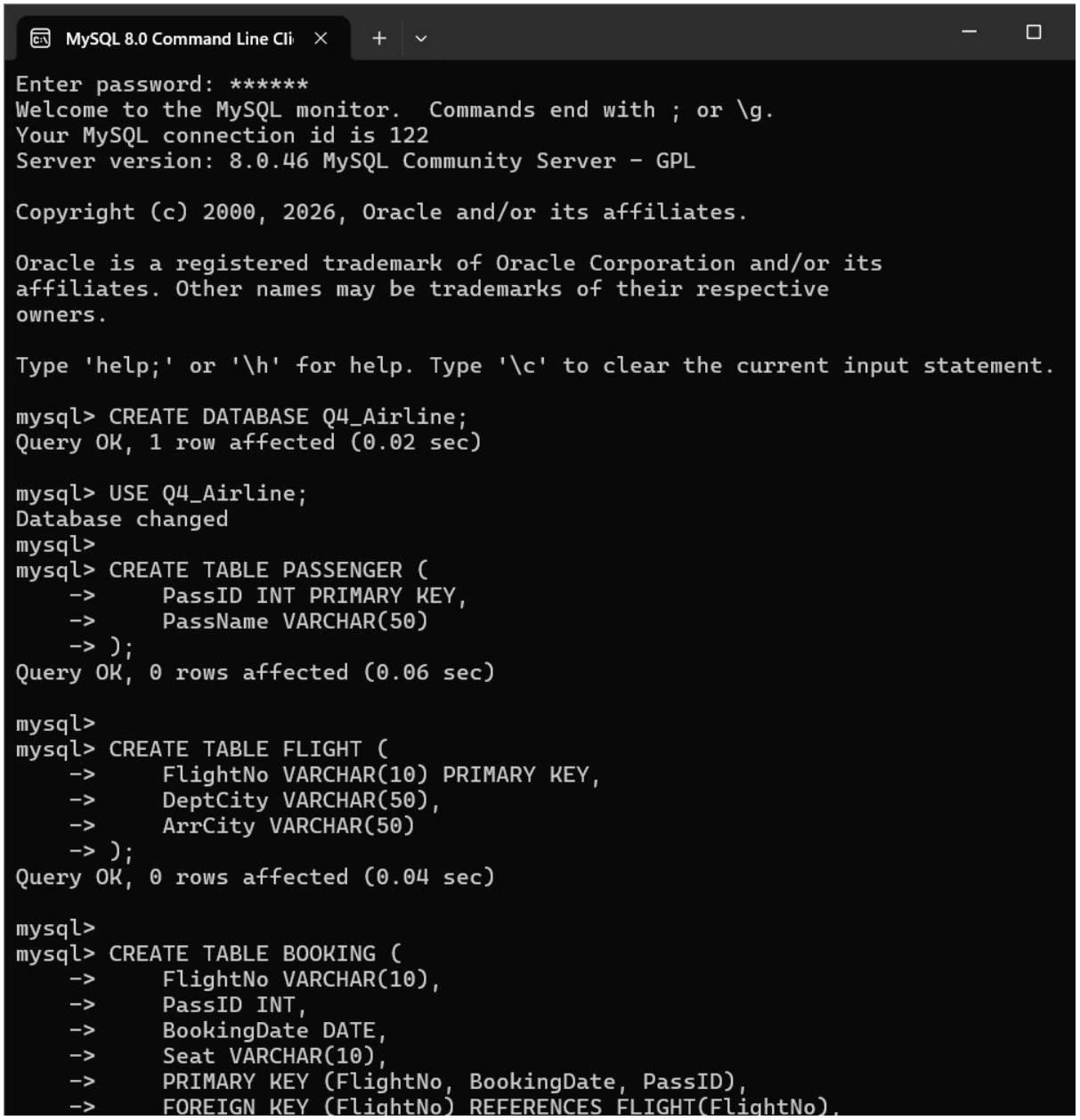
BOOKING(FlightNo, Date, PassID, Seat)

BCNF

In each relation, the determinant is a candidate key:

- PassID is key of PASSENGER.
- FlightNo is key of FLIGHT under the stated assumption.
- (FlightNo, Date, PassID) is key of BOOKING.

Therefore, the decomposed relations satisfy BCNF.


MySQL 8.0 Command Line Cli
Enter password: ******
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 122
Server version: 8.0.46 MySQL Community Server - GPL
Copyright (c) 2000, 2026, Oracle and/or its affiliates.
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
mysql> CREATE DATABASE Q4_Airline;
Query OK, 1 row affected (0.02 sec)
mysql> USE Q4_Airline;
Database changed
mysql>
mysql> CREATE TABLE PASSENGER (
    ->   PassID INT PRIMARY KEY,
    ->   PassName VARCHAR(50)
    ->);
Query OK, 0 rows affected (0.06 sec)
mysql>
mysql> CREATE TABLE FLIGHT (
    ->   FlightNo VARCHAR(10) PRIMARY KEY,
    ->   DeptCity VARCHAR(50),
    ->   ArrCity VARCHAR(50)
    ->);
Query OK, 0 rows affected (0.04 sec)
mysql>
mysql> CREATE TABLE BOOKING (
    ->   FlightNo VARCHAR(10),
    ->   PassID INT,
    ->   BookingDate DATE,
    ->   Seat VARCHAR(10),
    ->   PRIMARY KEY (FlightNo, BookingDate, PassID),
    ->   FOREIGN KEY (FlightNo) REFERENCES FLIGHT(FlightNo),

MySQL 8.0 Command Line Cli

```
    ->     FOREIGN KEY (PassID) REFERENCES PASSENGER(PassID)
    -> );
Query OK, 0 rows affected (0.06 sec)

mysql> INSERT INTO PASSENGER VALUES
    -> (1,'Aishwarya'),
    -> (2,'Priya');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO FLIGHT VALUES
    -> ('AI101','Chennai','Delhi'),
    -> ('AI202','Mumbai','Bangalore');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO BOOKING VALUES
    -> ('AI101',1,'2026-08-11','12A'),
    -> ('AI101',2,'2026-08-11','12B');
Query OK, 2 rows affected (0.02 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM PASSENGER;
+--------+-----------+
| PassID | PassName  |
+--------+-----------+
|      1 | Aishwarya |
|      2 | Priya     |
+--------+-----------+
2 rows in set (0.00 sec)

mysql> SELECT * FROM FLIGHT;
+----------+----------+-----------+
| FlightNo | DeptCity | ArrCity   |
+----------+----------+-----------+
| AI101    | Chennai  | Delhi     |
| AI202    | Mumbai   | Bangalore |
+----------+----------+-----------+
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM BOOKING;
+----------+--------+-------------+------+
| FlightNo | PassID | BookingDate | Seat |
+----------+--------+-------------+------+
| AI101    |      1 | 2026-08-11  | 12A  |
| AI101    |      2 | 2026-08-11  | 12B  |
+----------+--------+-------------+------+
2 rows in set (0.00 sec)

mysql> SELECT B.FlightNo, P.PassID, P.PassName,
    ->        B.Seat, F.DeptCity, F.ArrCity, B.BookingDate
    -> FROM BOOKING B
    -> JOIN PASSENGER P ON B.PassID = P.PassID
    -> JOIN FLIGHT F ON B.FlightNo = F.FlightNo;
+----------+--------+-----------+------+----------+---------+-------------+
| FlightNo | PassID | PassName  | Seat | DeptCity | ArrCity | BookingDate |
+----------+--------+-----------+------+----------+---------+-------------+
| AI101    |      1 | Aishwarya | 12A  | Chennai  | Delhi   | 2026-08-11  |
| AI101    |      2 | Priya     | 12B  | Chennai  | Delhi   | 2026-08-11  |
+----------+--------+-----------+------+----------+---------+-------------+
2 rows in set (0.01 sec)
```

Q5. HR System – Normalize to 3NF

Consider an HR relation:

EMPLOYEE(EmpID, EmpName, DeptID, DeptName, ManagerID, ManagerName, Location)

Functional Dependencies

1. EmpID → EmpName, DeptID
2. DeptID → DeptName, ManagerID, Location
3. ManagerID → ManagerName

Transitive Dependencies

From:

EmpID → DeptID

and

DeptID → DeptName, ManagerID, Location

we get:

EmpID → DeptName, ManagerID, Location

Also:

ManagerID → ManagerName

Therefore:

EmpID → ManagerName

is a **transitive dependency**.

3NF Decomposition

EMPLOYEE

EMPLOYEE(EmpID, EmpName, DeptID)

DEPARTMENT

DEPARTMENT(DeptID, DeptName, ManagerID, Location)

MANAGER

MANAGER(ManagerID, ManagerName)

Result

Transitive dependencies are removed and the schema satisfies 3NF.


MySQL 8.0 Command Line Cli
Enter password: ******
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 123
Server version: 8.0.46 MySQL Community Server - GPL
Copyright (c) 2000, 2026, Oracle and/or its affiliates.
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
mysql> CREATE DATABASE Q5_HR;
Query OK, 1 row affected (0.02 sec)
mysql> USE Q5_HR;
Database changed
mysql>
mysql> CREATE TABLE MANAGER (
    ->   ManagerID INT PRIMARY KEY,
    ->   ManagerName VARCHAR(50)
    ->);
Query OK, 0 rows affected (0.07 sec)
mysql>
mysql> CREATE TABLE DEPARTMENT (
    ->   DeptID INT PRIMARY KEY,
    ->   DeptName VARCHAR(50),
    ->   ManagerID INT,
    ->   Location VARCHAR(50),
    ->   FOREIGN KEY (ManagerID) REFERENCES MANAGER(ManagerID)
    ->);
Query OK, 0 rows affected (0.06 sec)
mysql>
mysql> CREATE TABLE EMPLOYEE (
    ->   EmpID INT PRIMARY KEY,
    ->   EmpName VARCHAR(50),
    ->   DeptID INT,
    ->   FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID)

MySQL 8.0 Command Line Cli

```
    -> );
Query OK, 0 rows affected (0.05 sec)

mysql> INSERT INTO MANAGER VALUES
    -> (1,'Raj'),
    -> (2,'Kumar');
Query OK, 2 rows affected (0.02 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO DEPARTMENT VALUES
    -> (10,'IT',1,'Chennai'),
    -> (20,'HR',2,'Bangalore');
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO EMPLOYEE VALUES
    -> (101,'Aishwarya',10),
    -> (102,'Priya',20);
Query OK, 2 rows affected (0.02 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM MANAGER;
+-----------+-------------+
| ManagerID | ManagerName |
+-----------+-------------+
|         1 | Raj         |
|         2 | Kumar       |
+-----------+-------------+
2 rows in set (0.00 sec)

mysql> SELECT * FROM DEPARTMENT;
+--------+----------+-----------+-----------+
| DeptID | DeptName | ManagerID | Location  |
+--------+----------+-----------+-----------+
|     10 | IT       |         1 | Chennai   |
|     20 | HR       |         2 | Bangalore |
+--------+----------+-----------+-----------+
2 rows in set (0.00 sec)
```

```
mysql> SELECT * FROM EMPLOYEE;
+-------+-----------+--------+
| EmpID | EmpName   | DeptID |
+-------+-----------+--------+
|   101 | Aishwarya |     10 |
|   102 | Priya     |     20 |
+-------+-----------+--------+
2 rows in set (0.00 sec)

mysql> SELECT E.EmpID, E.EmpName,
    ->        D.DeptName, M.ManagerName, D.Location
    -> FROM EMPLOYEE E
    -> JOIN DEPARTMENT D ON E.DeptID = D.DeptID
    -> JOIN MANAGER M ON D.ManagerID = M.ManagerID;
+-------+-----------+----------+-------------+-----------+
| EmpID | EmpName   | DeptName | ManagerName | Location  |
+-------+-----------+----------+-------------+-----------+
|   101 | Aishwarya | IT       | Raj         | Chennai   |
|   102 | Priya     | HR       | Kumar       | Bangalore |
+-------+-----------+----------+-------------+-----------+
2 rows in set (0.04 sec)
```

Q6. Library – Identify MVDs and Decompose to 4NF

Given Relation

LIBRARY(MemberID, MemberName, BookID, Title, Author, Genre, BorrowDate)

Assume a member can borrow multiple books and independently have multiple genres/interests associated with their library profile.

The important independent multivalued relationship can be modeled as:

MemberID →→ BookID

If multiple independent values exist for another attribute set, storing them together causes unnecessary combinations.

4NF Decomposition

Separate the independent relationships:

MEMBER

MEMBER(MemberID, MemberName)

BOOK

BOOK(BookID, Title, Author, Genre)

BORROW

BORROW(MemberID, BookID, BorrowDate)

Why?

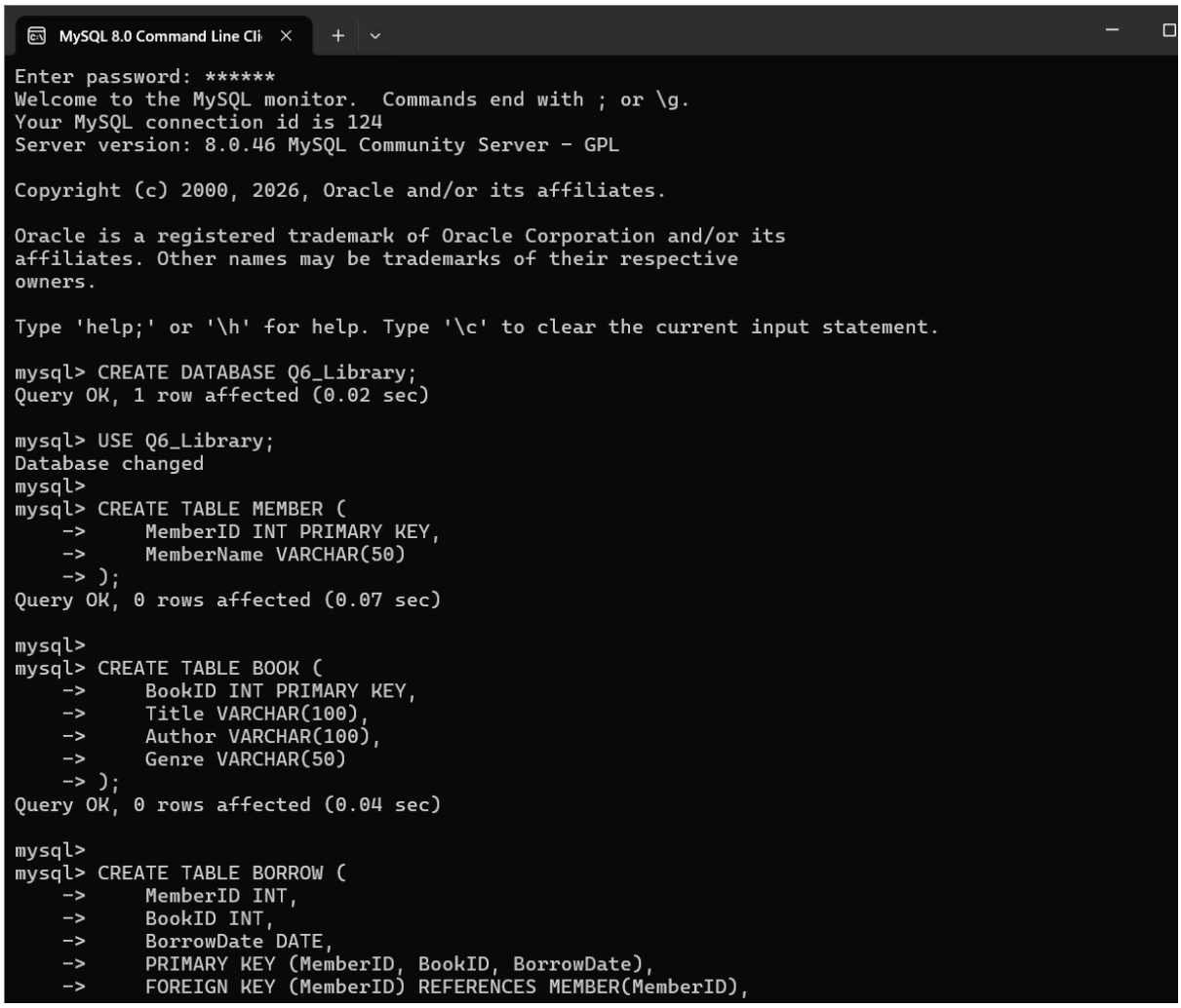
The original relation repeats:

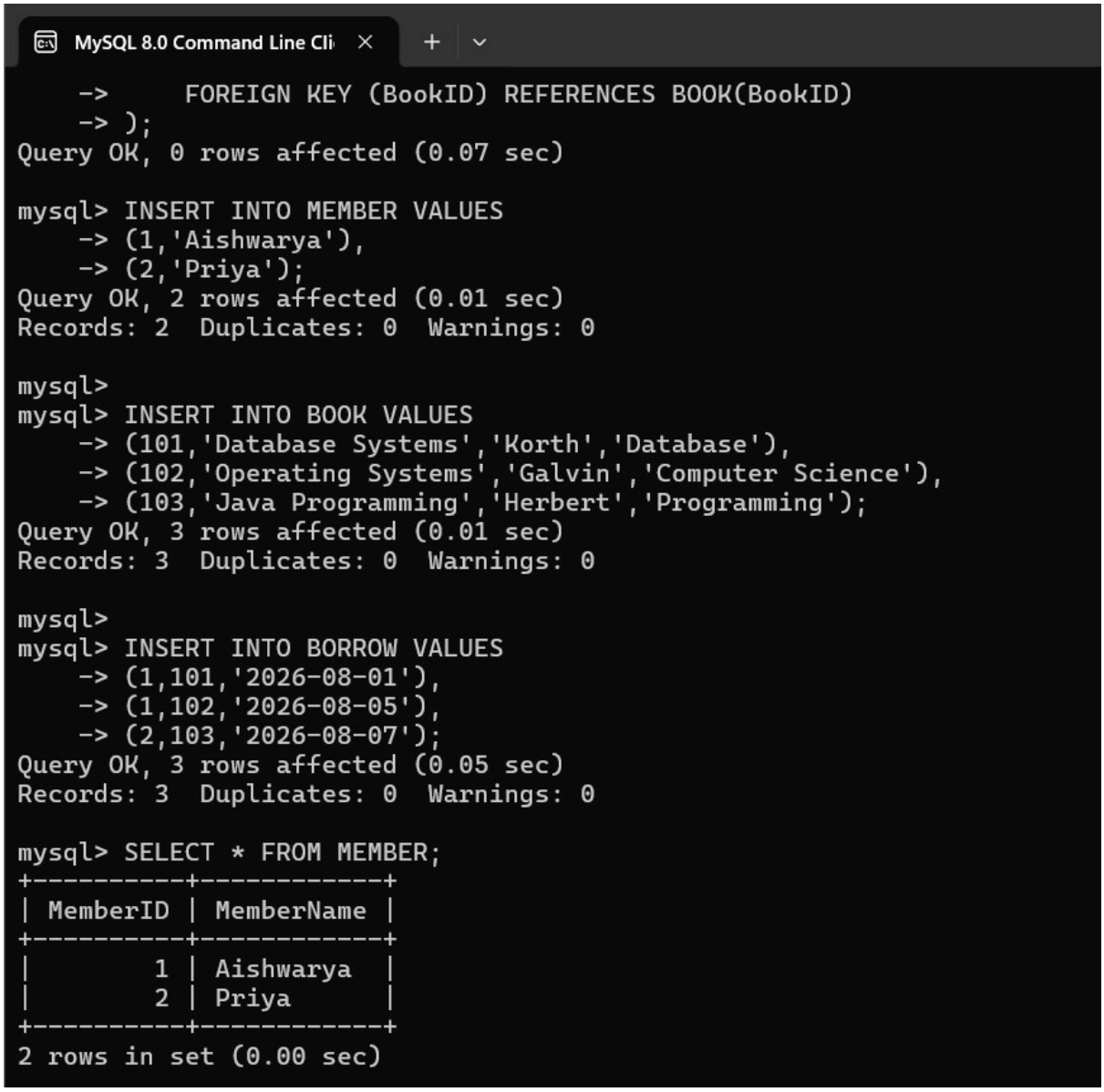
- Member details
- Book details
- Borrowing details

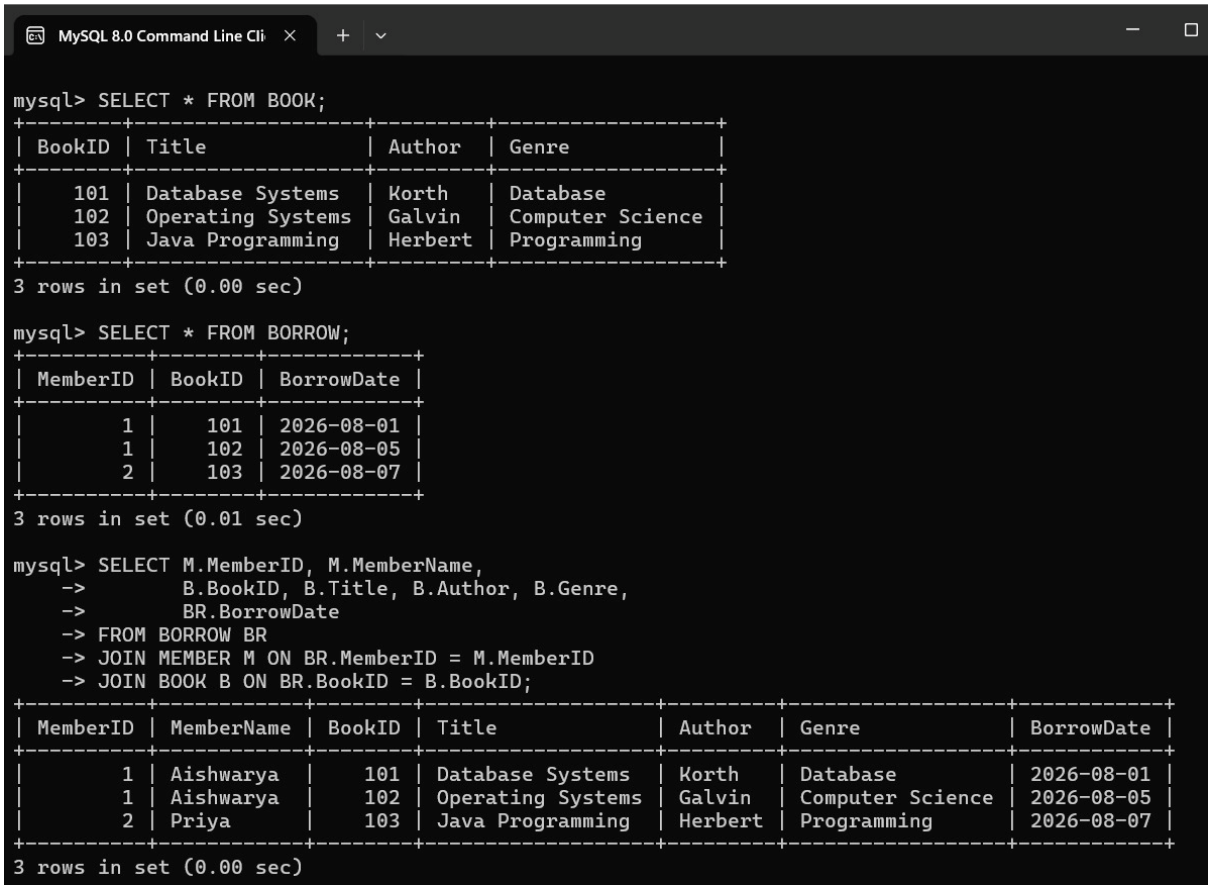
Separating them prevents unnecessary combinations and redundancy.

Result

The decomposed relations satisfy the required 4NF structure for the independent multivalued relationships.




MySQL 8.0 Command Line Cli
-> FOREIGN KEY (BookID) REFERENCES BOOK(BookID)
->);
Query OK, 0 rows affected (0.07 sec)
mysql> INSERT INTO MEMBER VALUES
-> (1,'Aishwarya'),
-> (2,'Priya');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0
mysql>
mysql> INSERT INTO BOOK VALUES
-> (101,'Database Systems','Korth','Database'),
-> (102,'Operating Systems','Galvin','Computer Science'),
-> (103,'Java Programming','Herbert','Programming');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0
mysql>
mysql> INSERT INTO BORROW VALUES
-> (1,101,'2026-08-01'),
-> (1,102,'2026-08-05'),
-> (2,103,'2026-08-07');
Query OK, 3 rows affected (0.05 sec)
Records: 3 Duplicates: 0 Warnings: 0
mysql> SELECT * FROM MEMBER;
+----------+------------+
| MemberID | MemberName |
+----------+------------+
| 1 | Aishwarya |
| 2 | Priya |
+----------+------------+
2 rows in set (0.00 sec)



Q7. Telecom Billing – Keys and FDs

Consider the telecom relation:

BILLING(CustomerID, PhoneNo, CustomerName, PlanID, PlanName, CallID, CallDate, Duration, Amount)

Functional Dependencies

1. CustomerID → CustomerName
2. PhoneNo → CustomerID, PlanID
3. PlanID → PlanName
4. CallID → PhoneNo, CallDate, Duration
5. (CallID, PhoneNo) → Amount

Candidate Keys

Under these assumptions:

CallID can identify a call record.

Therefore:

Candidate Key = CallID

If CallID is not globally unique and is only unique per phone number, then:

(CallID, PhoneNo) becomes the candidate key.

Primary Key

Choose:

CallID

as the primary key if it is globally unique.

Foreign Keys

- CustomerID references CUSTOMER.
- PlanID references PLAN.
- PhoneNo references the subscriber/phone entity.

Important FDs

CustomerID → CustomerName

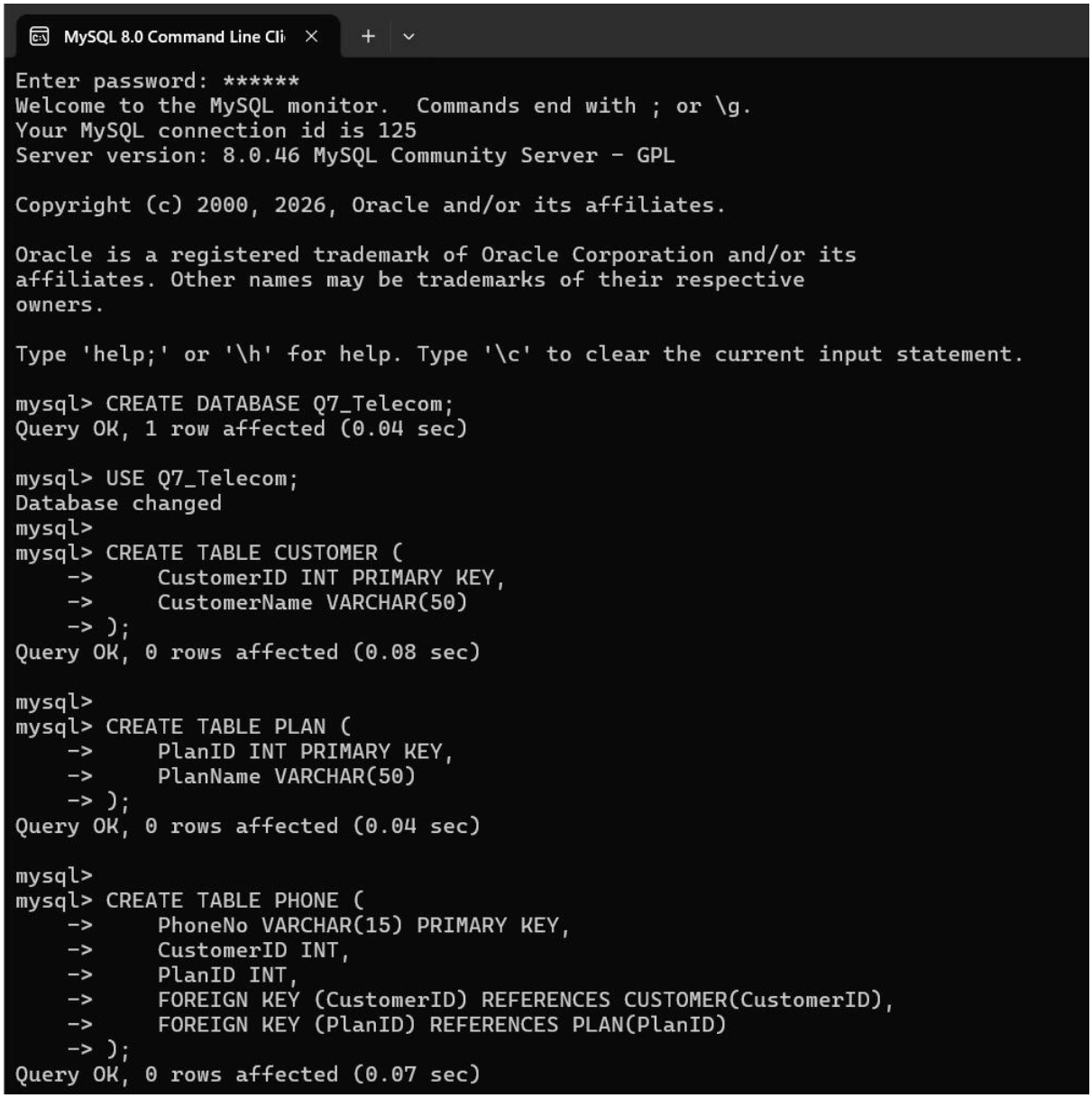
PhoneNo → CustomerID, PlanID

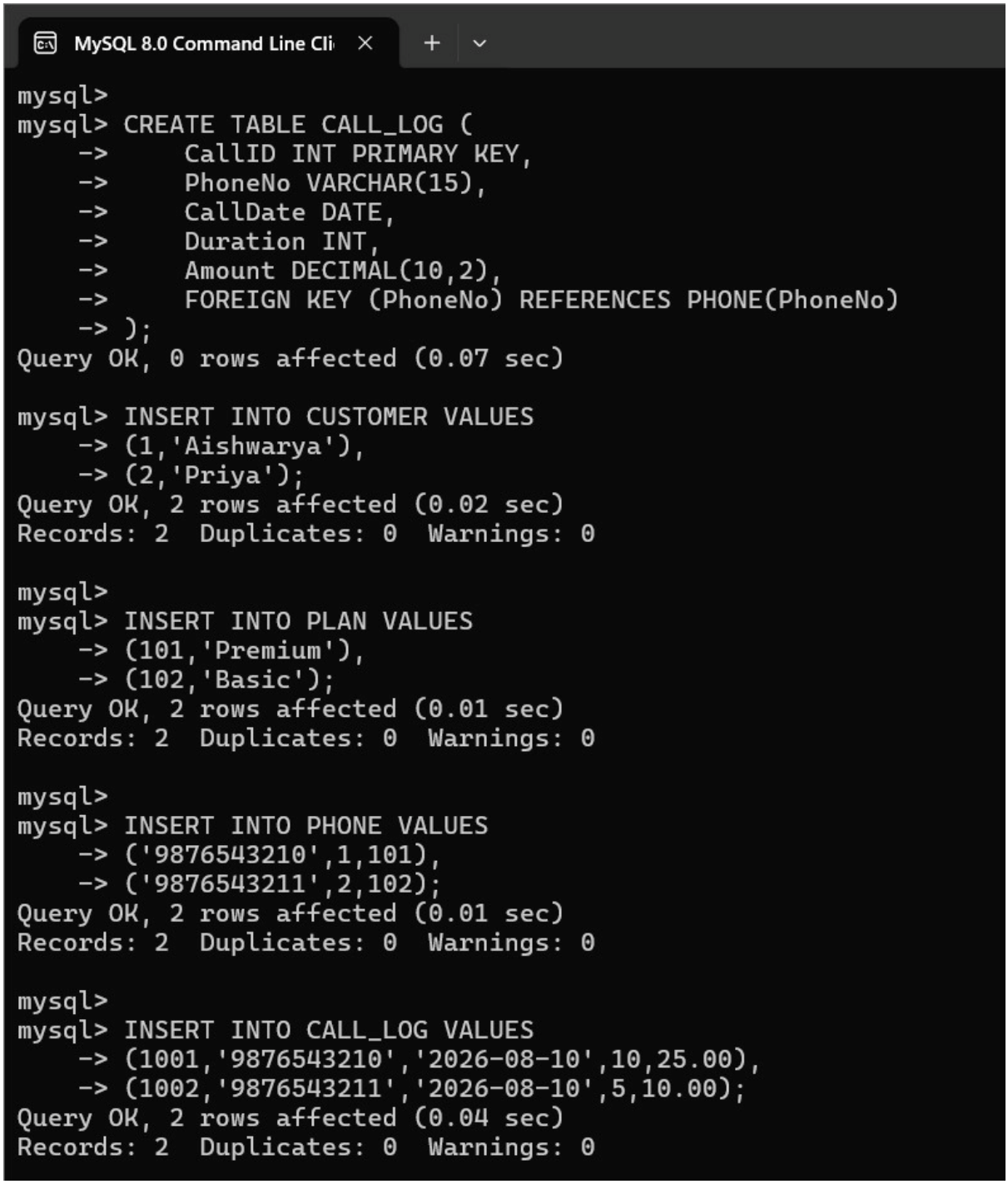
PlanID → PlanName

CallID → PhoneNo, CallDate, Duration

Result

Identifying keys and FDs before normalization helps remove redundancy and maintain data integrity.


MySQL 8.0 Command Line Cli
Enter password: ******
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 125
Server version: 8.0.46 MySQL Community Server - GPL
Copyright (c) 2000, 2026, Oracle and/or its affiliates.
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
mysql> CREATE DATABASE Q7_Telecom;
Query OK, 1 row affected (0.04 sec)
mysql> USE Q7_Telecom;
Database changed
mysql>
mysql> CREATE TABLE CUSTOMER (
    ->   CustomerID INT PRIMARY KEY,
    ->   CustomerName VARCHAR(50)
    ->);
Query OK, 0 rows affected (0.08 sec)
mysql>
mysql> CREATE TABLE PLAN (
    ->   PlanID INT PRIMARY KEY,
    ->   PlanName VARCHAR(50)
    ->);
Query OK, 0 rows affected (0.04 sec)
mysql>
mysql> CREATE TABLE PHONE (
    ->   PhoneNo VARCHAR(15) PRIMARY KEY,
    ->   CustomerID INT,
    ->   PlanID INT,
    ->   FOREIGN KEY (CustomerID) REFERENCES CUSTOMER(CustomerID),
    ->   FOREIGN KEY (PlanID) REFERENCES PLAN(PlanID)
    ->);
Query OK, 0 rows affected (0.07 sec)


MySQL 8.0 Command Line Cli
mysql>
mysql> CREATE TABLE CALL_LOG (
    ->   CallID INT PRIMARY KEY,
    ->   PhoneNo VARCHAR(15),
    ->   CallDate DATE,
    ->   Duration INT,
    ->   Amount DECIMAL(10,2),
    ->   FOREIGN KEY (PhoneNo) REFERENCES PHONE(PhoneNo)
    ->);
Query OK, 0 rows affected (0.07 sec)
mysql> INSERT INTO CUSTOMER VALUES
    -> (1,'Aishwarya'),
    -> (2,'Priya');
Query OK, 2 rows affected (0.02 sec)
Records: 2 Duplicates: 0 Warnings: 0
mysql>
mysql> INSERT INTO PLAN VALUES
    -> (101,'Premium'),
    -> (102,'Basic');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0
mysql>
mysql> INSERT INTO PHONE VALUES
    -> ('9876543210',1,101),
    -> ('9876543211',2,102);
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0
mysql>
mysql> INSERT INTO CALL_LOG VALUES
    -> (1001,'9876543210','2026-08-10',10,25.00),
    -> (1002,'9876543211','2026-08-10',5,10.00);
Query OK, 2 rows affected (0.04 sec)
Records: 2 Duplicates: 0 Warnings: 0

MySQL 8.0 Command Line Cli

```
mysql> SELECT * FROM CUSTOMER;
+------------+--------------+
| CustomerID | CustomerName |
+------------+--------------+
|          1 | Aishwarya    |
|          2 | Priya        |
+------------+--------------+
2 rows in set (0.04 sec)

mysql> SELECT * FROM PLAN;
+--------+----------+
| PlanID | PlanName |
+--------+----------+
|    101 | Premium  |
|    102 | Basic    |
+--------+----------+
2 rows in set (0.00 sec)

mysql> SELECT * FROM PHONE;
+------------+------------+--------+
| PhoneNo    | CustomerID | PlanID |
+------------+------------+--------+
| 9876543210 |          1 |    101 |
| 9876543211 |          2 |    102 |
+------------+------------+--------+
2 rows in set (0.00 sec)

mysql> SELECT * FROM CALL_LOG;
+--------+------------+------------+----------+--------+
| CallID | PhoneNo    | CallDate   | Duration | Amount |
+--------+------------+------------+----------+--------+
|   1001 | 9876543210 | 2026-08-10 |       10 |  25.00 |
|   1002 | 9876543211 | 2026-08-10 |        5 |  10.00 |
+--------+------------+------------+----------+--------+
2 rows in set (0.04 sec)
```

```
mysql> SELECT C.CustomerID, C.CustomerName,
    ->        P.PhoneNo, PL.PlanName,
    ->        CL.CallID, CL.CallDate,
    ->        CL.Duration, CL.Amount
    -> FROM CALL_LOG CL
    -> JOIN PHONE P ON CL.PhoneNo = P.PhoneNo
    -> JOIN CUSTOMER C ON P.CustomerID = C.CustomerID
    -> JOIN PLAN PL ON P.PlanID = PL.PlanID;
+------------+--------------+------------+----------+--------+------------+----------+--------+
| CustomerID | CustomerName | PhoneNo    | PlanName | CallID | CallDate   | Duration | Amount |
+------------+--------------+------------+----------+--------+------------+----------+--------+
|          1 | Aishwarya    | 9876543210 | Premium  |   1001 | 2026-08-10 |       10 |  25.00 |
|          2 | Priya        | 9876543211 | Basic    |   1002 | 2026-08-10 |        5 |  10.00 |
+------------+--------------+------------+----------+--------+------------+----------+--------+
2 rows in set (0.01 sec)
```

Q8. Retail Inventory – Complete Normalization

Given Relation

PRODUCT(PID, PName, SupplierID, SupplierName, Category, Price, Warehouse)

Functional Dependencies

1. PID → PName, SupplierID, Category, Price, Warehouse
2. SupplierID → SupplierName

Candidate Key

PID

because:

PID+ = {PID, PName, SupplierID, SupplierName, Category, Price, Warehouse}

1NF

All attributes contain atomic values.

2NF

Since the key is a single attribute PID, there are **no partial dependencies**.

Thus, it is in 2NF.

3NF

There is a transitive dependency:

PID → SupplierID

and

SupplierID → SupplierName

Therefore:

PID → SupplierName

Decomposition

PRODUCT

PRODUCT(PID, PName, SupplierID, Category, Price, Warehouse)

SUPPLIER

SUPPLIER(SupplierID, SupplierName)

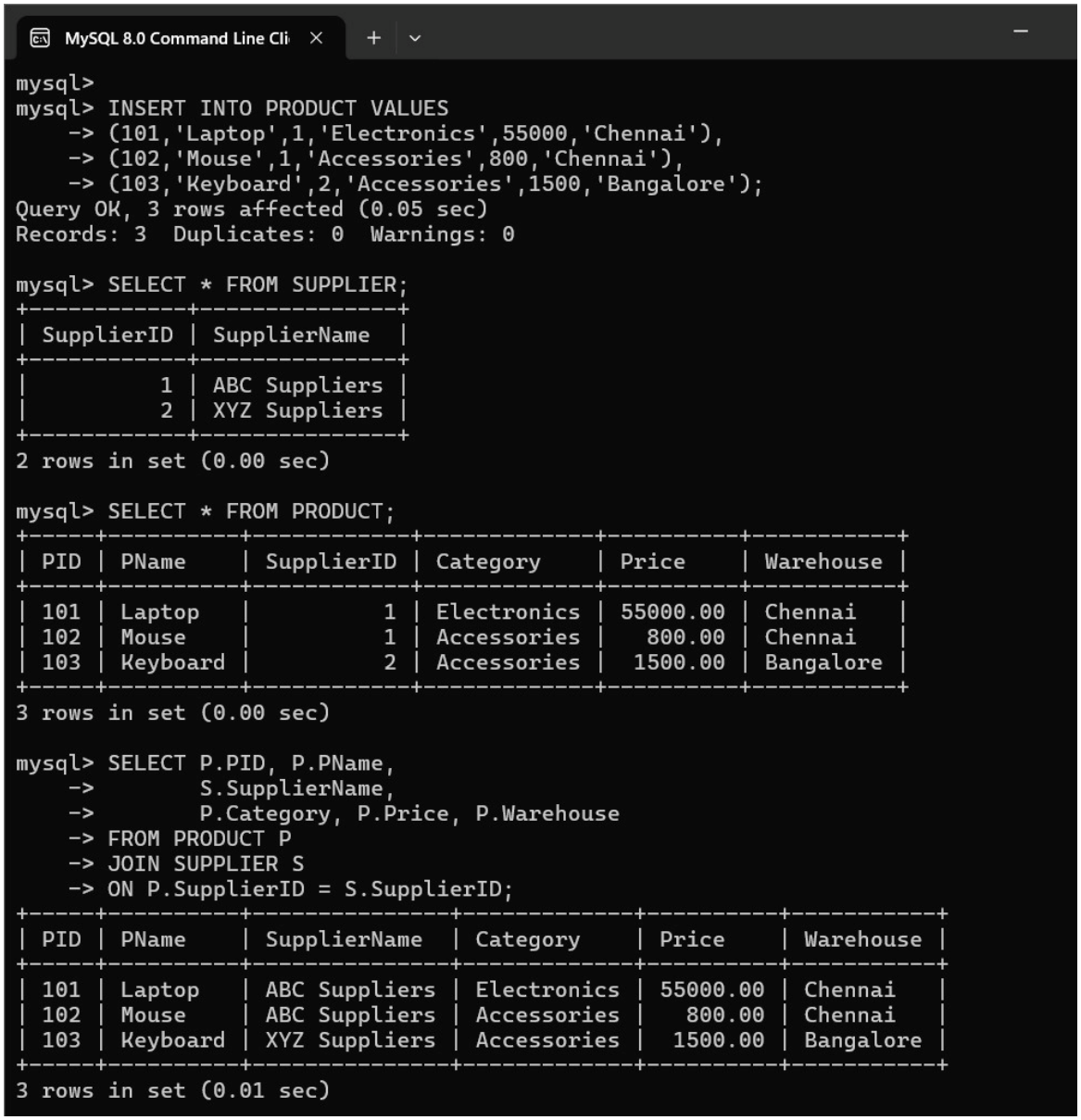
Final Result

The database is normalized to **3NF**.

If Warehouse represents multiple warehouses per product, it should additionally be separated:

PRODUCT_WAREHOUSE(PID, Warehouse)

This prevents multivalued redundancy.



Q9. School ERP – Fully Normalized up to BCNF

Consider a poorly designed schema:

SCHOOL(StudentID, StudentName, ClassID, ClassName, TeacherID, TeacherName, SubjectID, SubjectName, Grade)

Functional Dependencies

1. StudentID → StudentName
2. ClassID → ClassName, TeacherID
3. TeacherID → TeacherName
4. SubjectID → SubjectName
5. (StudentID, SubjectID) → Grade

Problems

The original relation has:

- Repeated student information
- Repeated teacher information
- Repeated subject information
- Update anomaly
- Insertion anomaly
- Deletion anomaly
- Transitive dependencies

BCNF Decomposition

STUDENT

STUDENT(StudentID, StudentName, ClassID)

CLASS

CLASS(ClassID, ClassName, TeacherID)

TEACHER

TEACHER(TeacherID, TeacherName)

SUBJECT

SUBJECT(SubjectID, SubjectName)

RESULT

RESULT(StudentID, SubjectID, Grade)

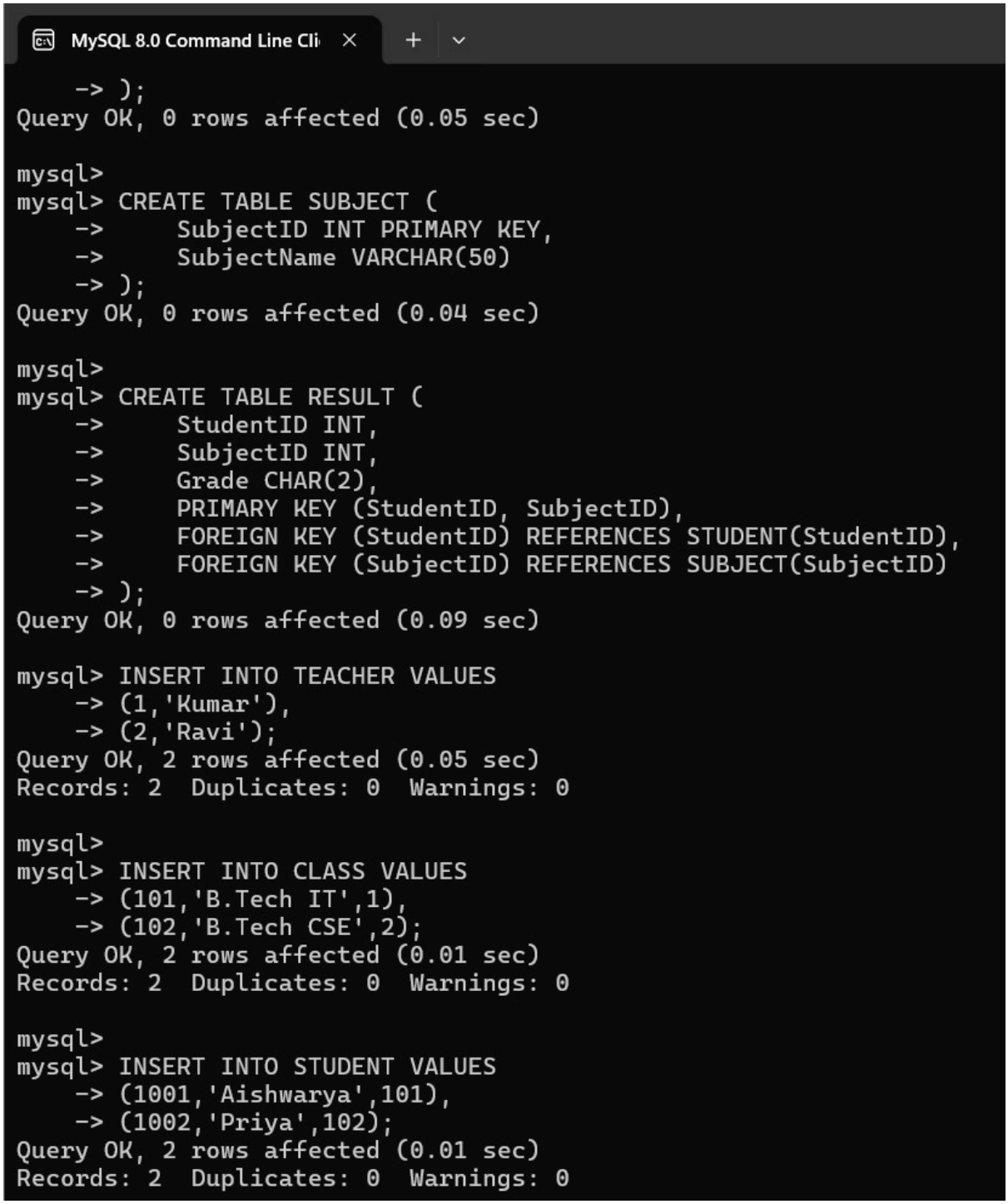
Keys

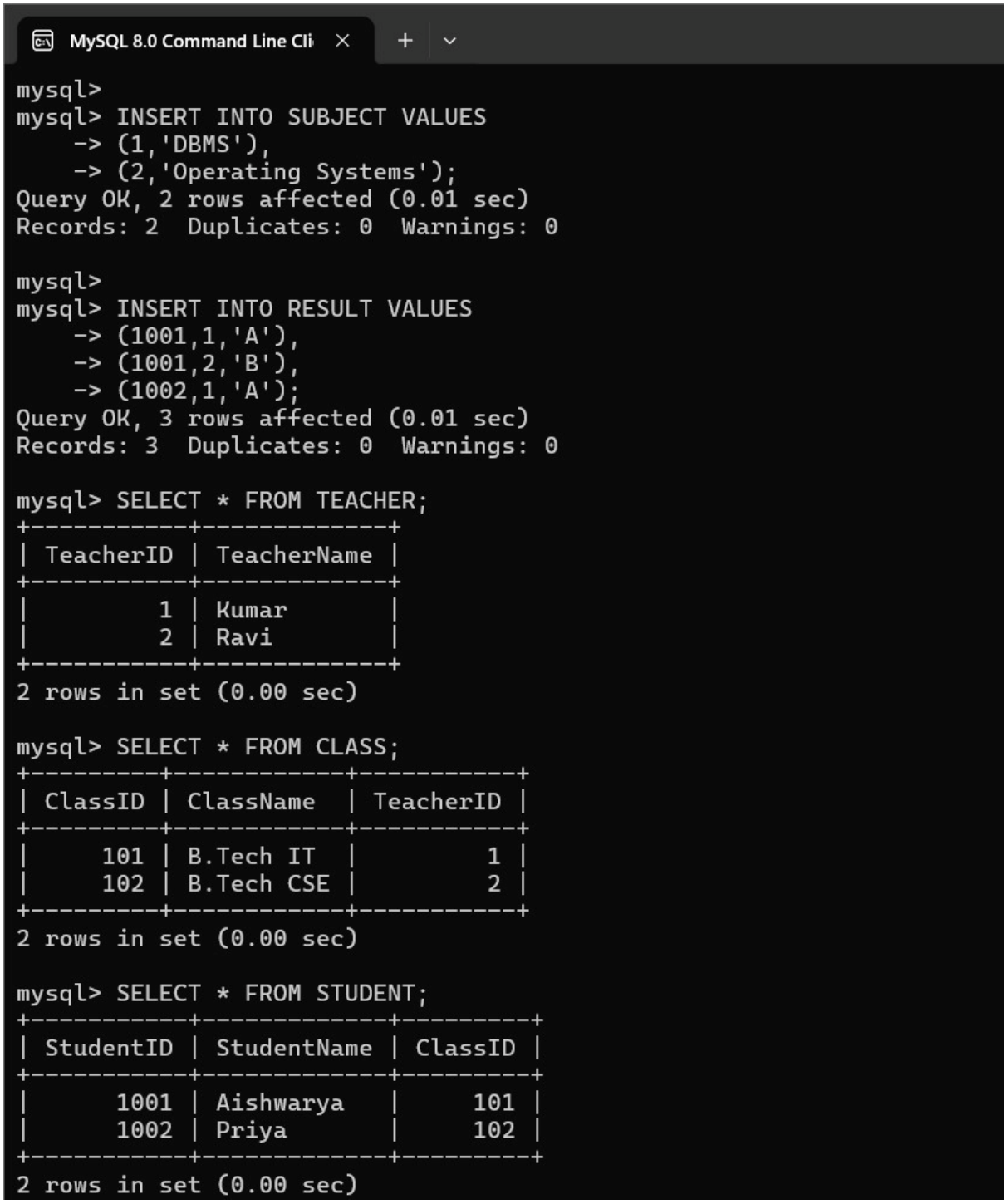
Relation	**Primary Key**
STUDENT	StudentID
CLASS	ClassID
TEACHER	TeacherID
SUBJECT	SubjectID
RESULT	(StudentID, SubjectID)

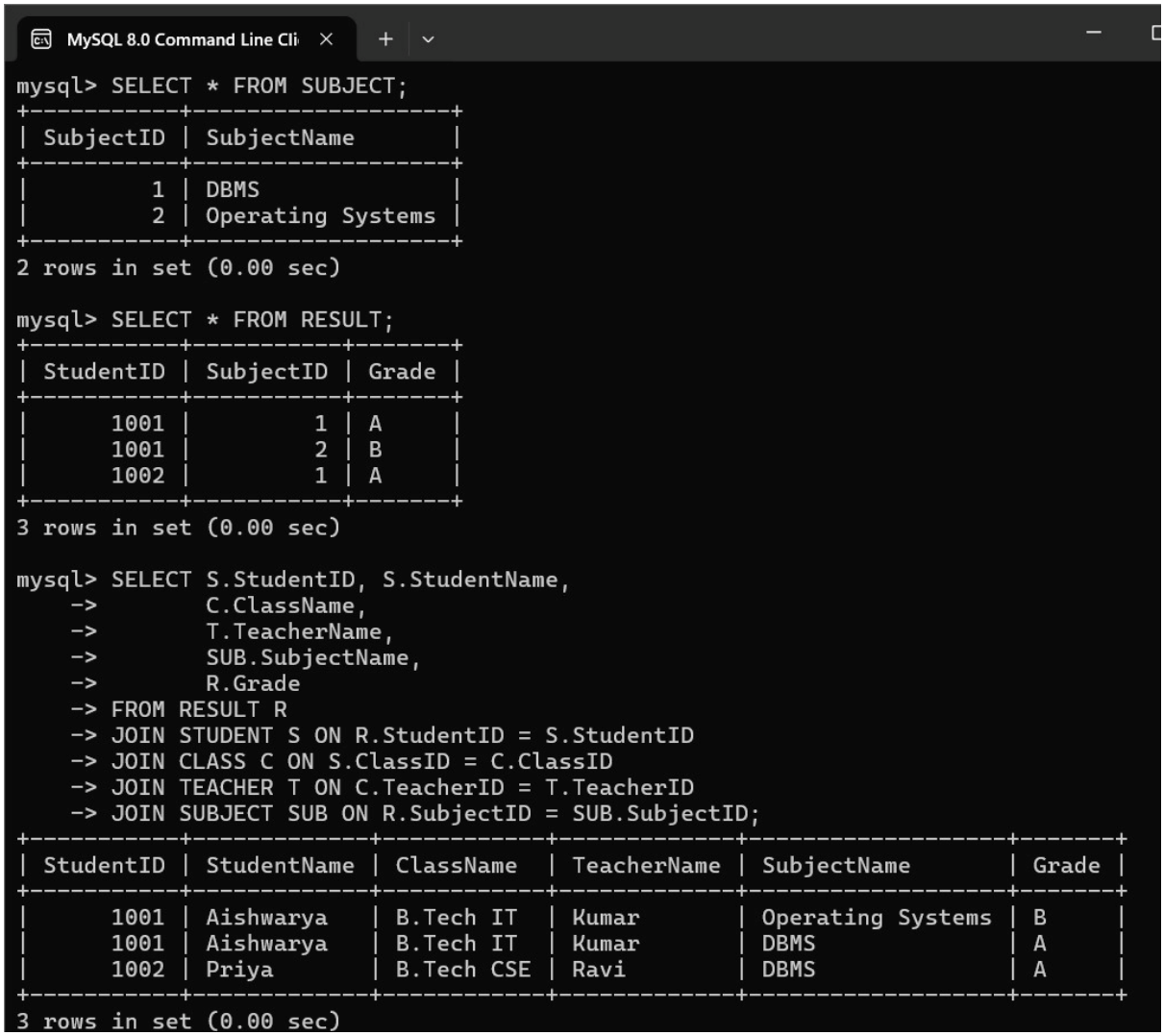
Every determinant in these relations is a candidate/superkey.

Result

The schema is normalized up to **BCNF**, reducing redundancy and maintaining referential integrity.


MySQL 8.0 Command Line Cli
->);
Query OK, 0 rows affected (0.05 sec)
mysql>
mysql> CREATE TABLE SUBJECT (
-> SubjectID INT PRIMARY KEY,
-> SubjectName VARCHAR(50)
->);
Query OK, 0 rows affected (0.04 sec)
mysql>
mysql> CREATE TABLE RESULT (
-> StudentID INT,
-> SubjectID INT,
-> Grade CHAR(2),
-> PRIMARY KEY (StudentID, SubjectID),
-> FOREIGN KEY (StudentID) REFERENCES STUDENT(StudentID),
-> FOREIGN KEY (SubjectID) REFERENCES SUBJECT(SubjectID)
->);
Query OK, 0 rows affected (0.09 sec)
mysql> INSERT INTO TEACHER VALUES
-> (1,'Kumar'),
-> (2,'Ravi');
Query OK, 2 rows affected (0.05 sec)
Records: 2 Duplicates: 0 Warnings: 0
mysql>
mysql> INSERT INTO CLASS VALUES
-> (101,'B.Tech IT',1),
-> (102,'B.Tech CSE',2);
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0
mysql>
mysql> INSERT INTO STUDENT VALUES
-> (1001,'Aishwarya',101),
-> (1002,'Priya',102);
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0


MySQL 8.0 Command Line Cli
mysql>
mysql> INSERT INTO SUBJECT VALUES
-> (1,'DBMS'),
-> (2,'Operating Systems');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0
mysql>
mysql> INSERT INTO RESULT VALUES
-> (1001,1,'A'),
-> (1001,2,'B'),
-> (1002,1,'A');
Query OK, 3 rows affected (0.01 sec)
Records: 3 Duplicates: 0 Warnings: 0
mysql> SELECT * FROM TEACHER;
+-----------+-------------+
| TeacherID | TeacherName |
+-----------+-------------+
| 1 | Kumar |
| 2 | Ravi |
+-----------+-------------+
2 rows in set (0.00 sec)
mysql> SELECT * FROM CLASS;
+---------+-----------+-----------+
| ClassID | ClassName | TeacherID |
+---------+-----------+-----------+
| 101 | B.Tech IT | 1 |
| 102 | B.Tech CSE | 2 |
+---------+-----------+-----------+
2 rows in set (0.00 sec)
mysql> SELECT * FROM STUDENT;
+-----------+-------------+---------+
| StudentID | StudentName | ClassID |
+-----------+-------------+---------+
| 1001 | Aishwarya | 101 |
| 1002 | Priya | 102 |
+-----------+-------------+---------+
2 rows in set (0.00 sec)



Q10. Movie Streaming – Join Dependency and 5NF

Consider:

STREAM(MovieID, ActorID, LanguageID)

Suppose movie participation has three independent relationships:

- Movie–Actor
- Movie–Language
- Actor–Language

The original relation may store redundant combinations.

Join Dependency

The relation can have the join dependency:

(MovieID, ActorID), (MovieID, LanguageID), (ActorID, LanguageID)

Decomposition

MOVIE_ACTOR

MOVIE_ACTOR(MovieID, ActorID)

MOVIE_LANGUAGE

MOVIE_LANGUAGE(MovieID, LanguageID)

ACTOR_LANGUAGE

ACTOR_LANGUAGE(ActorID, LanguageID)

5NF

The relation is decomposed so that the non-trivial join dependency is represented by smaller relations.

Lossless-Join Verification

Reconstruct:

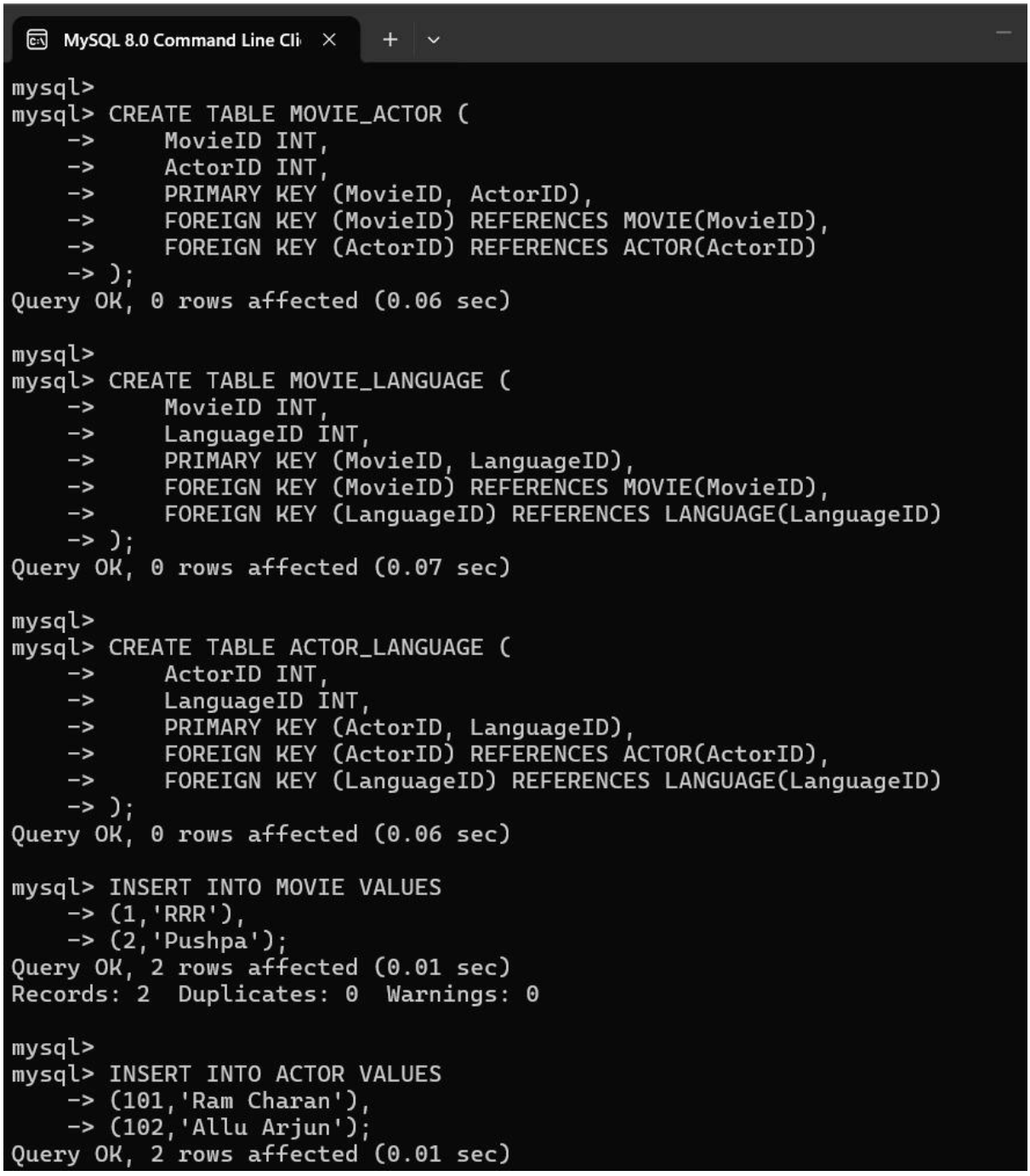
MOVIE_ACTOR ⋈ MOVIE_LANGUAGE ⋈ ACTOR_LANGUAGE

If the join produces exactly the original valid tuples without spurious tuples, the decomposition is **lossless**.

Result

The decomposition removes redundancy caused by the join dependency and satisfies **5NF (Project-Join Normal Form)** under the stated business rules.


MySQL 8.0 Command Line Cli
Enter password: ******
Welcome to the MySQL monitor. Commands end with ; or \g.
Your MySQL connection id is 128
Server version: 8.0.46 MySQL Community Server - GPL
Copyright (c) 2000, 2026, Oracle and/or its affiliates.
Oracle is a registered trademark of Oracle Corporation and/or its
affiliates. Other names may be trademarks of their respective
owners.
Type 'help;' or '\h' for help. Type '\c' to clear the current input statement.
mysql> CREATE DATABASE Q9_School;
Query OK, 1 row affected (0.01 sec)
mysql> USE Q9_School;
Database changed
mysql>
mysql> CREATE TABLE TEACHER (
    ->   TeacherID INT PRIMARY KEY,
    ->   TeacherName VARCHAR(50)
    ->);
Query OK, 0 rows affected (0.05 sec)
mysql>
mysql> CREATE TABLE CLASS (
    ->   ClassID INT PRIMARY KEY,
    ->   ClassName VARCHAR(50),
    ->   TeacherID INT,
    ->   FOREIGN KEY (TeacherID) REFERENCES TEACHER(TeacherID)
    ->);
Query OK, 0 rows affected (0.05 sec)
mysql>
mysql> CREATE TABLE STUDENT (
    ->   StudentID INT PRIMARY KEY,
    ->   StudentName VARCHAR(50),
    ->   ClassID INT,
    ->   FOREIGN KEY (ClassID) REFERENCES CLASS(ClassID)
    ->);


MySQL 8.0 Command Line Cli
mysql>
mysql> CREATE TABLE MOVIE_ACTOR (
    ->   MovieID INT,
    ->   ActorID INT,
    ->   PRIMARY KEY (MovieID, ActorID),
    ->   FOREIGN KEY (MovieID) REFERENCES MOVIE(MovieID),
    ->   FOREIGN KEY (ActorID) REFERENCES ACTOR(ActorID)
    ->);
Query OK, 0 rows affected (0.06 sec)
mysql>
mysql> CREATE TABLE MOVIE_LANGUAGE (
    ->   MovieID INT,
    ->   LanguageID INT,
    ->   PRIMARY KEY (MovieID, LanguageID),
    ->   FOREIGN KEY (MovieID) REFERENCES MOVIE(MovieID),
    ->   FOREIGN KEY (LanguageID) REFERENCES LANGUAGE(LanguageID)
    ->);
Query OK, 0 rows affected (0.07 sec)
mysql>
mysql> CREATE TABLE ACTOR_LANGUAGE (
    ->   ActorID INT,
    ->   LanguageID INT,
    ->   PRIMARY KEY (ActorID, LanguageID),
    ->   FOREIGN KEY (ActorID) REFERENCES ACTOR(ActorID),
    ->   FOREIGN KEY (LanguageID) REFERENCES LANGUAGE(LanguageID)
    ->);
Query OK, 0 rows affected (0.06 sec)
mysql> INSERT INTO MOVIE VALUES
    -> (1,'RRR'),
    -> (2,'Pushpa');
Query OK, 2 rows affected (0.01 sec)
Records: 2 Duplicates: 0 Warnings: 0
mysql>
mysql> INSERT INTO ACTOR VALUES
    -> (101,'Ram Charan'),
    -> (102,'Allu Arjun');
Query OK, 2 rows affected (0.01 sec)

MySQL 8.0 Command Line Cli

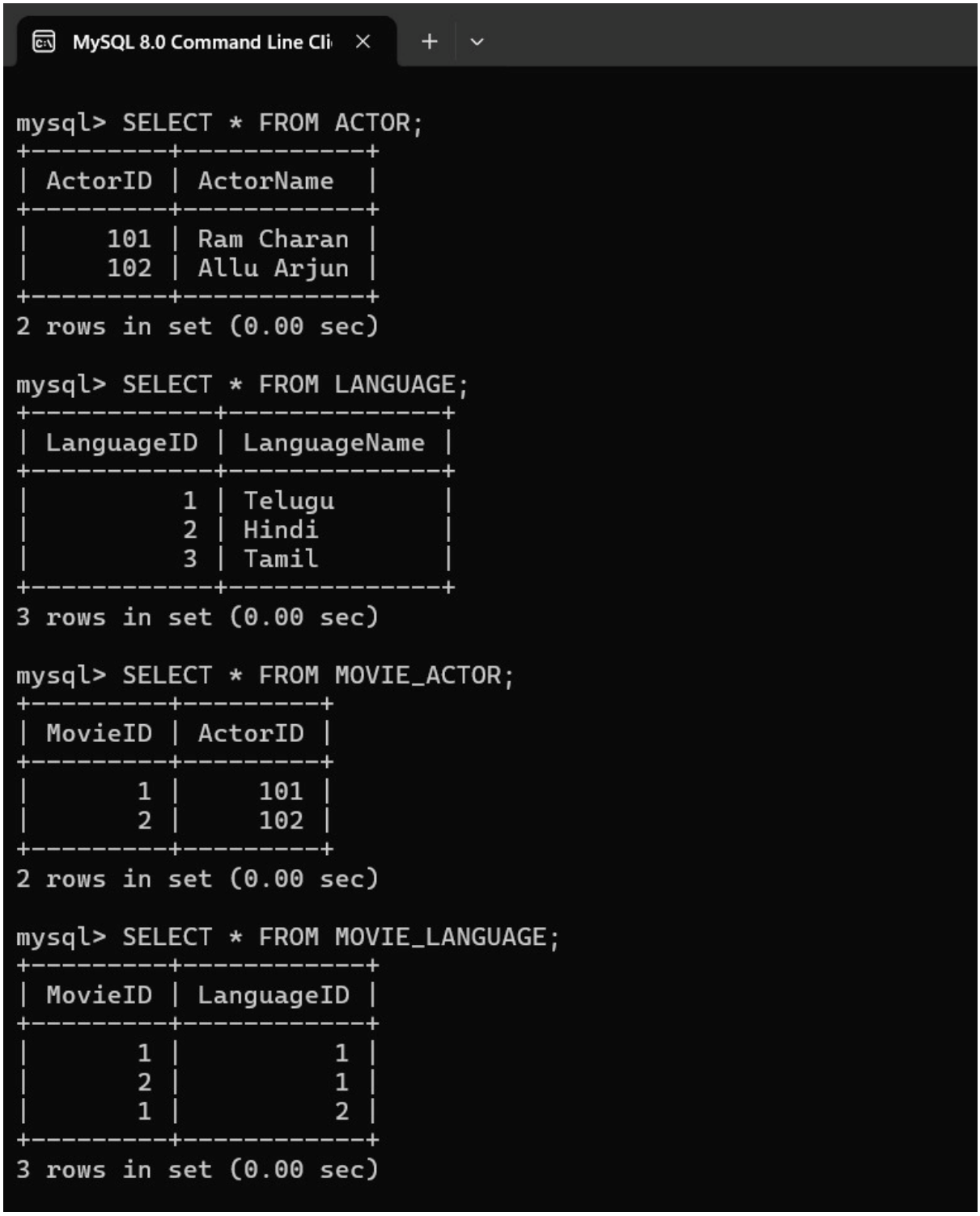
```
mysql>
mysql> INSERT INTO LANGUAGE VALUES
    -> (1,'Telugu'),
    -> (2,'Hindi'),
    -> (3,'Tamil');
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO MOVIE_ACTOR VALUES
    -> (1,101),
    -> (2,102);
Query OK, 2 rows affected (0.01 sec)
Records: 2  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO MOVIE_LANGUAGE VALUES
    -> (1,1),
    -> (1,2),
    -> (2,1);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql>
mysql> INSERT INTO ACTOR_LANGUAGE VALUES
    -> (101,1),
    -> (101,2),
    -> (102,1);
Query OK, 3 rows affected (0.01 sec)
Records: 3  Duplicates: 0  Warnings: 0

mysql> SELECT * FROM MOVIE;
+---------+-----------+
| MovieID | MovieName |
+---------+-----------+
|       1 | RRR       |
|       2 | Pushpa    |
+---------+-----------+
2 rows in set (0.00 sec)
```


MySQL 8.0 Command Line Cli
mysql> SELECT * FROM ACTOR;
+---------+------------+
| ActorID | ActorName |
+---------+------------+
| 101 | Ram Charan |
| 102 | Allu Arjun |
+---------+------------+
2 rows in set (0.00 sec)
mysql> SELECT * FROM LANGUAGE;
+------------+--------------+
| LanguageID | LanguageName |
+------------+--------------+
1	Telugu
2	Hindi
3	Tamil
+------------+--------------+
3 rows in set (0.00 sec)
mysql> SELECT * FROM MOVIE_ACTOR;
+---------+---------+
| MovieID | ActorID |
+---------+---------+
| 1 | 101 |
| 2 | 102 |
+---------+---------+
2 rows in set (0.00 sec)
mysql> SELECT * FROM MOVIE_LANGUAGE;
+---------+------------+
| MovieID | LanguageID |
+---------+------------+
1	1
2	1
1	2
+---------+------------+
3 rows in set (0.00 sec)

```
mysql> SELECT * FROM ACTOR_LANGUAGE;
+---------+------------+
| ActorID | LanguageID |
+---------+------------+
|     101 |          1 |
|     102 |          1 |
|     101 |          2 |
+---------+------------+
3 rows in set (0.00 sec)

mysql> SELECT M.MovieName,
    ->        A.ActorName,
    ->        L.LanguageName
    -> FROM MOVIE_ACTOR MA
    -> JOIN MOVIE M ON MA.MovieID = M.MovieID
    -> JOIN ACTOR A ON MA.ActorID = A.ActorID
    -> JOIN MOVIE_LANGUAGE ML ON M.MovieID = ML.MovieID
    -> JOIN LANGUAGE L ON ML.LanguageID = L.LanguageID;
+-----------+------------+--------------+
| MovieName | ActorName  | LanguageName |
+-----------+------------+--------------+
| RRR       | Ram Charan | Telugu       |
| RRR       | Ram Charan | Hindi        |
| Pushpa    | Allu Arjun | Telugu       |
+-----------+------------+--------------+
3 rows in set (0.04 sec)
```